

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра програмної інженерії

ЗВІТ
з лабораторної роботи № 4
«Рефакторинг коду та проведення Code Review за індустріальними
стандартами»

з дисципліни «Основи програмної інженерії»

Виконав: студент групи ПЗПІ-25-2 **Петров Данило**

Перевірив: **Гребенюк В'ячеслав**

Харків 2026

Опис використання інструментів штучного інтелекту

При виконанні роботи використовувався AI-асистент Claude (Anthropic).
Нижче наведено основні промпти та корекції студента.

1. Підготовка коду для Code Review

Промпт: «Напиши новий модуль `batch.py` для системи LocalSquash із типовими `code smells` (погане іменування, магічні числа, мертвий код, порушення SRP), але функціонально коректний — щоб був матеріал для рев'ю. Додай тести.»

Корекція: студент попросив підвищити цикломатичну складність функції, щоб приклад був показовим — у результаті СС доведено до B(7).

2. Рефакторинг за результатами рев'ю

Промпт: «За 6 зауваженнями з `code review` відрефактори `batch.py`: перейменуй, винеси магічні числа в константи, застосуй `Extract Method`, видали мертвий код. Після кожної зміни запускай тести.»

Корекція: після рефакторингу залишився один задовгий рядок (`ruff E501`) — студент вказав на нього, рядок скорочено, `ruff` доведено до 0.

3. Оформлення звіту

Промпт: «Збери `docx`-звіт за 7 розділами методички ЛР4: тема, таблиця рев'ю коду одногрупника, посилання на PR, рефакторинг БУЛО→СТАЛО→ЧОМУ, скрін тестів, метрики ДО/ПІСЛЯ, рефлексія. Times New Roman 14, інтервал 1.5.»

Усі змістовні рішення — які смели позначати, як саме рефакторити — приймалися студентом і верифіковані запуском тестів (44 passed) та лінтера.

1 Тема та мета лабораторної роботи

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета: набуття практичних навичок проведення Code Review через GitHub Pull Requests, ідентифікації та класифікації code smells, застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2 Результати Code Review коду одногрупника

Перевірявся модуль `group_booking.py` з проєкту `cinema-booking-lab` (система бронювання кінотеатру). Знайдено та задокументовано як inline-коментарі у Pull Request такі проблеми якості коду:

Таблиця 2.1 — Результати Code Review

№	Рядок коду	Проблема	Категорія	Рекомендація
1	<code># import datetime (p.3)</code>	Закоментований невикористаний import	Dead Code	Видалити — Git зберігає історію
2	<code>def book_group(ppl, s) (p.6)</code>	Імена <code>ppl, s, cb, lst, a, pr, nm, cnt</code> не відображають змісту	Poor Naming	<code>book_group(people, session), price, names, count</code>
3	<code>except: (p.18)</code>	Перехоплення всіх винятків без розбору	Bare Except (E722)	<code>except ValueError</code>
4	<code># logged = nm (p.24)</code>	Закоментований рядок-залишок	Dead Code	Видалити
5	<code>elif cnt > 10 (p.25-27)</code>	Гілка недосяжна: якщо <code>cnt > 10</code> , то <code>cnt > 5</code> вже True	Logic Bug / Unreachable	Перевірити спочатку <code>cnt > 10</code>
6	<code>total *= 0.9 / 0.8 (p.25-28)</code>	Пороги 5/10 і коефіцієнти 0.9/0.8 без пояснення	Magic Numbers	Винести в іменовані константи
7	<code>txt = txt + item (p.30-31)</code>	Конкатенація рядків у циклі; функція робить усе одразу	Long Method / SRP	<code>"\n".join()</code> ; розбити на функції

3 Посилання на Pull Request з коментарями Code Review

Рев'ю коду одногрупника (модуль `group_booking.py`):

<https://github.com/Liza1001/cinema-booking-lab/pull/1>

Рев'ю власного коду, отримане від одногрупника (модуль `batch.py`):

<https://github.com/ArtimanJustice/bse-lr1-petrov/pull/2>

4 Результати рефакторингу власного коду

За результатами рев'ю модуля batch.py (отримано 6 зауважень) виконано три рефакторинг-операції. Після кожної запускався тестовий набір — усі тести залишались зеленими.

Операція 1. Rename + Replace Magic Number with Constant

БУЛО:

```
def proc(lst, t, q):
    ...
    if t == 1:
        c = ImageCompressor(q)
```

СТАЛО:

```
IMAGE = 1
VIDEO = 2
AUDIO = 3

def compress_files(paths, media_type, setting):
    ...
    if media_type == IMAGE:
```

ЧОМУ: імена proc, lst, t, q не відображали призначення; числа 1/2/3 — магичні. Перейменування та винесення констант підвищили читабельність.

Операція 2. Extract Method

БУЛО: вибір компресора і стиснення були вкладені в основний цикл (if/elif t==1/2/3).

СТАЛО:

```
def _compress_one(path, media_type, setting):
    """Стискає один файл за типом медіа."""
    if media_type == IMAGE:
        return ImageCompressor(setting).compress_image(path, path + ".out")
    if media_type == VIDEO:
        return VideoCompressor(setting).compress_video(path, path + ".out")
    if media_type == AUDIO:
        return AudioCompressor(setting).compress_audio(path, path + ".out")
    return None
```

ЧОМУ: функція proc порушувала SRP і мала складність B(7). Винесення фабрики компресора знизило цикломатичну складність до A(4) та зробило код читабельнішим.

Операція 3. Remove Dead Code + Simplify (None, style)

БУЛО:

```
# import os
    if r != None:
        total = total + r.original_size_mb
        # old = r.original_size_mb * 0.5
```

СТАЛО:

```
    if result is not None:
        total += result.original_size_mb
```

ЧОМУ: видалено мертвий код; порівняння `!= None` замінено на `is not None` (PEP 8); конкатенацію підсумкового рядка замінено на f-string, а `total = total + ...` на `+=`.

5 Звіт регресійного тестування

Після всіх рефакторинг-операцій запущено повний тестовий набір (38 тестів ЛРЗ + 6 тестів модуля batch.py). Усі 44 тести проходять — зовнішня поведінка коду збережена.

```
$ pytest test_compressor.py test_batch.py -v

===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.3
collected 44 items

test_compressor.py ..... [ 63%]
test_compressor.py ..... [ 86%]
test_batch.py::test_single_image PASSED [ 88%]
test_batch.py::test_multiple_images PASSED [ 90%]
test_batch.py::test_video PASSED [ 93%]
test_batch.py::test_audio PASSED [ 95%]
test_batch.py::test_empty_list PASSED [ 97%]
test_batch.py::test_unknown_type PASSED [100%]

===== 44 passed in 0.05s =====
```

Рисунок 5.1 — Результат запуску pytest (44 passed)

6 Порівняння метрик ДО / ПІСЛЯ

У таблиці 6.1 наведено порівняння метрик якості модуля batch.py до та після рефакторингу. Лінер — Ruff, метрики складності — Radon.

Таблиця 6.1 — Метрики до та після рефакторингу

Метрика	ДО	ПІСЛЯ
Ruff warnings (статичний аналіз)	6	0
SonarLint issues	6	0
Цикломатична складність (max)	B (7)	A (4)
Maintainability Index	A (66.6)	A (64.5)
Тести (pass / total)	44 / 44	44 / 44

Кількість зауважень статичного аналізу скоротилась з 6 до 0. Цикломатична складність основної функції знизилась з B(7) до A(4) завдяки декомпозиції. Кількість тестів, що проходять, не змінилась (44/44) — рефакторинг не порушив поведінки.

```
# ===== ДО рефакторингу (batch.py) =====

$ ruff check batch.py --statistics
2  PLR2004  magic-value-comparison
1  E501     line-too-long
1  E711     none-comparison
1  RET504   unnecessary-assign
1  SIM108   if-else-block-instead-of-if-exp
Found 6 errors.

$ radon cc batch.py -s
F 6:0 proc - B (7)
```

Рисунок 6.1 — Метрики ДО рефакторингу (ruff 6 errors, CC B(7))

```
# ===== ПІСЛЯ рефакторингу (batch.py) =====

$ ruff check batch.py --statistics
All checks passed!

$ radon cc batch.py -s
F 9:0 _compress_one - A (4)
F 20:0 compress_files - A (4)

$ pytest test_compressor.py test_batch.py -q
44 passed in 0.05s
```

Рисунок 6.2 — Метрики ПІСЛЯ рефакторингу (ruff 0, CC A(4), 44 passed)

7 Підсумкова рефлексія

Найціннішим досвідом стало усвідомлення, що Code Review — це не пошук помилок, а діалог про якість коду. Формулюючи зауваження до коду одногрупника, я навчився чітко класифікувати code smells за категоріями та пропонувати конкретні рекомендації, а не абстрактну критику. Отримавши зауваження до власного коду, я на практиці побачив, як магичні числа й погані імена ускладнюють розуміння навіть невеликого модуля. Найкорисніша знахідка — недосяжна гілка elif у коді одногрупника: статичний лінтер її не виявив, лише уважне ручне рев'ю. Регресійне тестування після кожного кроку дало впевненість, що рефакторинг безпечний. Загалом робота показала, що поєднання ручного рев'ю, статичного аналізу та автотестів дає найкращий результат для якості ПЗ.